Video Aficionado

Liran Ma, Zhipeng Cai, and Wei Cheng

Copyright © 2019 Liran Ma, Texas Christian University; Zhipeng Cai, Georgia State University; Wei Cheng, University of Washington Tacoma

The development of this document is partially funded by the National Science Foundation's CyberTraining program under award No. OAC-1829553. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at `http://www.gnu.org/licenses/fdl.html`.

Introduction

The purpose of this lab is to understand and exploit the vulnerabilities of "open-access" mobile data for privacy breaches. Typically, mobile devices such as smartphones have a set of rigorous access control mechanisms (e.g., formal permission requests to access personal information) to protect private or otherwise sensitive user data. As a result, these control mechanisms prohibit a third party (e.g., an app) from directly identifying private user information. However, not all mobile data is strictly prohibited for access, e.g., power consumption of a mobile device is considered to be non-sensitive. Subsequently, it is possible to infer private information from open-access data that is available to all apps.

Contents

1 Overview **2**

2 Lab Prep **2**

3 Video Inference **5**

4 Lab Tasks **7**

5 General Questions **7**

1 Overview

Users enjoy the convenience of watching videos on smart devices on the go. However, video watching records can be exposed without user's knowledge and be exploited to infer private information. In this lab, we will experience a side-channel attack system, named *video aficionado*, which can identify video watching information without violating any access control policies on Android.

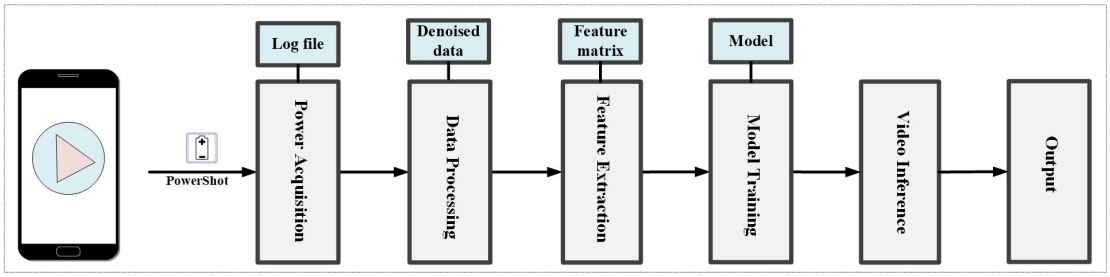


Figure 1: The framework.

The system only needs to collect power consumption data of a video playing app, which does not require explicit user permission. The collected data is cleaned and identified by a Multi-Layer Perceptron (MLP) trained classifier. Fig. 1 illustrates the work flow of *video aficionado*. The flow contains five steps: power acquisition, data processing, feature extraction, model training, and video inference. In this lab, you will use and modify the system to indirectly infer videos displayed on a mobile device without violating the control mechanisms.

2 Lab Prep

2.1 Tools

To perform lab activities, we will need to download/install/use the following tools:

- Weka: Machine Learning software in Java (available at `https://www.cs.waikato.ac.nz/ml/weka/`) that includes the necessary machine learning algorithms needed to infer the videos being watched.
- MATLAB: A a proprietary multi-paradigm programming language and numeric computing environment developed by MathWorks. It will be used to extract features (saving them into .dat files) from raw power consumption data of mobile devices. It is available to students in many universities with a site license.
- Python (`https://www.python.org/`) will be used to generate .arff files that can be used by Weka.

- PowerRecord: an App that collects raw power consumption data from mobile devices. In this lab, we provide collected power consumption data for your convenience. For interested learners, the App can be accessed via `https://github.com/luoziyuan97/PowerRecord`.

2.2 Data

All data for this lab is available at `http://csfaculty.tcu.edu/ma/eureka/Video-Aficionado/data.zip`. Download data.zip and extract the zip file to a local folder. The extracted folder contains the following sub-folders and files.

- The "power data" folder
  - Example raw power measurements of 10 videos (.txt).
  - Code to extract features from power measurements of each video including "Data_process_CDF.m", "Feature_extraction.m", and "temp.m".
  - The extracted feature data (.dat).
- The "feature data" folder
  - Code to combine multiple video feature data including "resh.m", "resh1.m", and "temp.m".
  - Combined training and test feature data (w=xx_training x.dat and w=xx_test x.dat).
- The "arff data" folder
  - Code to add names of features and labels for the combined training and test feature data.
  - Code to transfer .dat files to .arff files.
  - Coded files (.arff) for Weka.

2.3 Feature Extraction

First, enter each video's power file in the "power data" folder, run the "temp.m" file to get the corresponding feature file. Code in temp.m will invoke data pre-processing (CDF and EMA de-noising processes) and feature extraction functions (Feature_extraction()) to obtain the feature data for each video.

The CDF is the integral of probability density function (PDF) describing the probability distribution of a random variable X. The definition of CDF is given by the following equation:

$$F_X(x) = P(X \leq x), \tag{1}$$

where the right-hand part of (1) denotes the probability the value of the random variable X is equal to or less than x.

We exploit an EMA-based filtering process to cut off these small fluctuations. Moving Average (MA) is an approach to evaluate data points through generating a series of averages for various subsets inside the full data set. There are several types of MA including Simple Moving Average (SMA), Weighted Moving Average (WMA), Exponential Moving Average (EMA) and etc. Here, we utilize EMA that is primarily based on (2):

$$S_t = \begin{cases} Y_1, & t = 1 \\ a \cdot Y_t + (1-a) \cdot S_{t-1}, & t > 1 \end{cases} \tag{2}$$

where Y_t denotes the original data point, S_t represents the data point after the EMA-based filtering process, and a is a parameter whose value lies between 0 to 1.

file_sma() in "Data_process_CDF.m" file records the de-noised data after the EMA-based process (the parameter a is represented as m/n in the file and can be modified as you wish). Towards the feature extraction step, the cleansed data is processed in units of sliding windows (the length of the sliding windows can be manually adjusted). Then 30 features are extracted for each sliding window and are collected into a .dat file. The names and specific computation of these 30 features ("feature(,1),...,feature(,30)") are shown in the "Feature_extraction.m" file.

Copy all the obtained feature files from the previous step into the "feature data" folder, run the "temp.m" file to get the combined training and test data set. Specifically, we firstly add labels (see the 31st column in the training and testing data files i.e. w=xx_training/test.dat) for each sliding window's feature data, the labels for all the sliding windows of one video are set as the same.

Then, conduct the temp.m file invoking resh() function to randomly select 90% rows in each feature data record of each video (each video containing 10 records) and combined them as the training data set (w=XX_training.dat); the remaining data are combined as the test data set (w=XX_test.dat). This step can be executed multiple times and corresponding training and test datasets can be obtained. (temp.m also can invoke the resh1() function random selecting 9 records of each video to make the training data set).

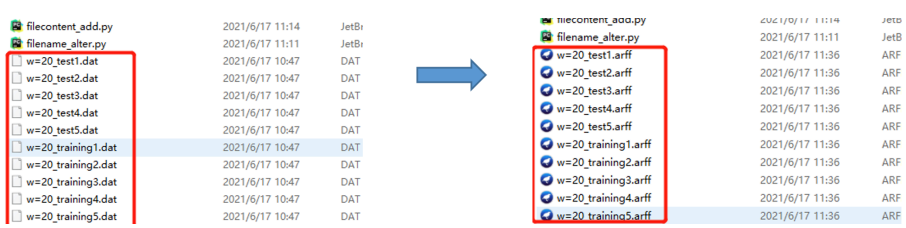


Figure 2: Covert dat to arff

Copy all the obtained .dat files from the previous step into the "arff data" folder, run the "file-

name_alter.py" and "filecontent_add.py" in sequence, and get the .arff training and test data that can be directly utilized by Weka. Specifically, the .arff files are the same as the training/test.dat files except additionally adding the head parameters representing the names of the 30 features and labels.

3 Video Inference

In this section, we will conduct video inference based on power consumption features using Weka. Open Weka and click Explorer as shown in Fig. 3.

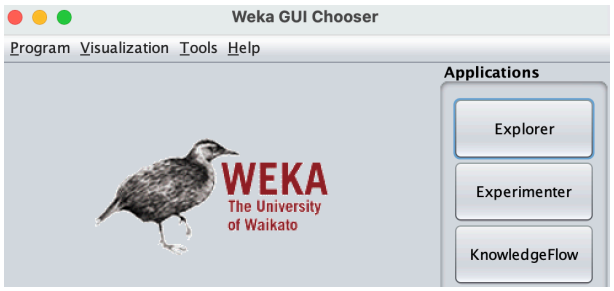


Figure 3: Explorer in Weka.

Next, under the Preprocess module, click "Open file". A window will pop up and select a training data set (in arff format). An example is shown in Fig. 4. The training data set will used to check new data sets (i.e., test data sets) to make a classification decision. The steps to obtain training data sets are explained in the previous section.

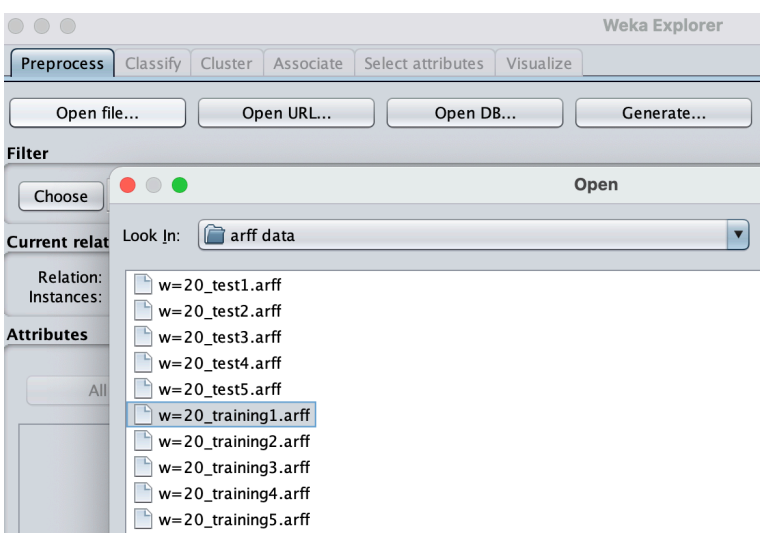


Figure 4: Choose a training data set.

Once the training data set is selected, we will need to select a test data set for classification. Switch

to the Classify module by clicking on the Classify button. Under this module, choose "Supplied test set" and select a test data set. An example is shown in Fig. 5.

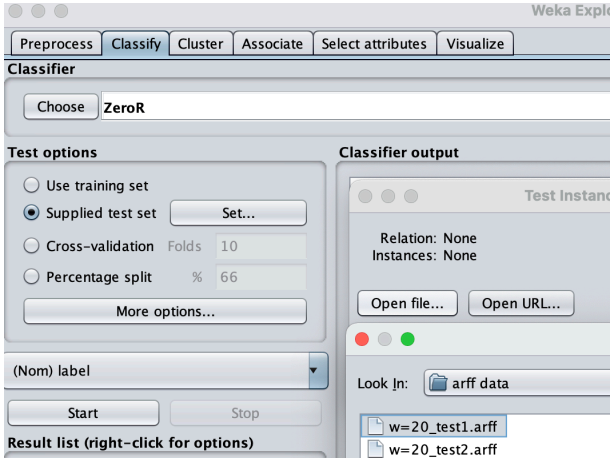


Figure 5: Choose a test data set for classification.

Next, choose a classification algorithm to perform video inference. An example is shown in Fig. 6, where RandomTree algorithm was selected to perform video inference.

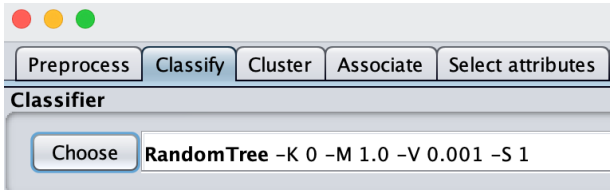
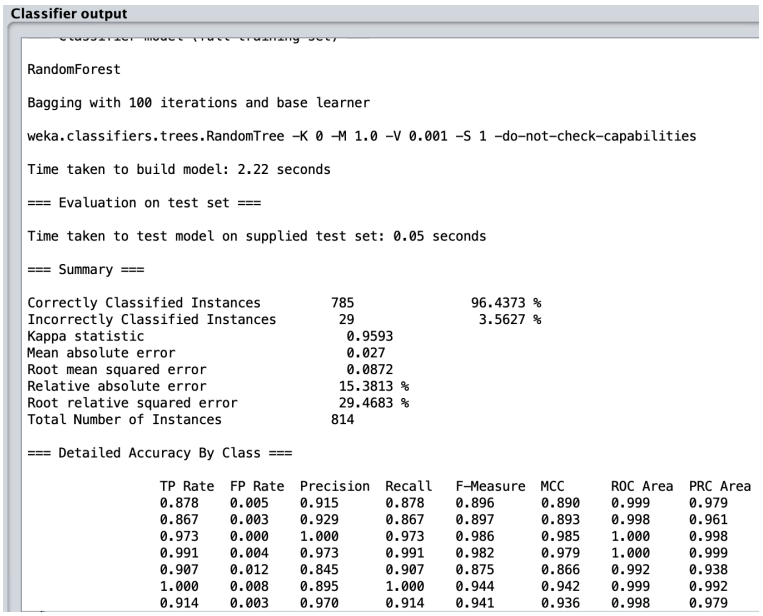


Figure 6: Choose a classification algorithm.

Example video inference results using RandomTree are given in Fig. 7. Experimental results in our paper published in IEEE Transactions on Mobile Computing (`https://ieeexplore.ieee.org/document/9298868`) demonstrate that our system can make a prediction with 74.5% accuracy on average for each 20-second power measurement segment out of 3918 segments collected from 20 videos. To the best of our knowledge, video aficionado is the first real-time power consumption-based video identification system on smart devices.



```
Classifier output

RandomForest

Bagging with 100 iterations and base learner

weka.classifiers.trees.RandomTree -K 0 -M 1.0 -V 0.001 -S 1 -do-not-check-capabilities

Time taken to build model: 2.22 seconds

=== Evaluation on test set ===

Time taken to test model on supplied test set: 0.05 seconds

=== Summary ===

Correctly Classified Instances         785               96.4373 %
Incorrectly Classified Instances        29                3.5627 %
Kappa statistic                          0.9593
Mean absolute error                      0.027
Root mean squared error                  0.0872
Relative absolute error                 15.3813 %
Root relative squared error             29.4683 %
Total Number of Instances              814

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area
                 0.878    0.005    0.915      0.878    0.896      0.890    0.999     0.979
                 0.867    0.003    0.929      0.867    0.897      0.893    0.998     0.961
                 0.973    0.000    1.000      0.973    0.986      0.985    1.000     0.998
                 0.991    0.004    0.973      0.991    0.982      0.979    1.000     0.999
                 0.907    0.012    0.845      0.907    0.875      0.866    0.992     0.938
                 1.000    0.008    0.895      1.000    0.944      0.942    0.999     0.992
                 0.914    0.003    0.970      0.914    0.941      0.936    0.998     0.979
```

Figure 7: classifier-output

4 Lab Tasks

4.0.1 Explore the following factors on the performance of classification results: number of sample per video, sliding window size, different features and their combinations.

4.0.2 Try applying other classification algorithms (e.g., decision tree, support vector machine, and neural networks). Show your results and explain them? Why they work or not work?

4.0.3 Explore other sources of open-data that can be exploited to infer private information. Show your results and justifications.

4.0.4 How could you prevent such types of attacks? Present and justify your solution.

5 General Questions

5.0.1 How long did it take you to complete this lab?

5.0.2 Are there any corrections that need to be made to this lab?

5.0.3 Any suggestions to make this lab better?